

Universidad de las Fuerzas Armadas ESPE

Investigacion sobre las Plataformas de Desarrollo Embebidas

Tema de investigación: Plataformas Embebidas

Integrantes:

Josue Chango

Jardel Maza

Alejandro Obando

NRC: 29703

Fecha:

29 de junio de 2026

Introducción

Este informe presenta una investigación comparativa de cinco plataformas de desarrollo embebidas ampliamente utilizadas en la industria y el ámbito educativo. Para cada una se describen sus características técnicas, arquitectura, memoria, interfaces de comunicación, entorno de desarrollo, lenguajes de programación, sistemas operativos soportados y aplicaciones más comunes. Las plataformas analizadas son Arduino, ESP32, STM32, Raspberry Pi y Nordic nRF52.

1. Arduino



Figura 1: Arduino

1.1. Descripción general

Arduino es una plataforma *open-source* de hardware y software creada por Arduino LLC, fundada por Massimo Banzi, David Cuartielles y otros, con sede entre Italia y Estados Unidos.

Se lanzó en 2005 con la primera placa Arduino NG, y está orientada a prototipos rápidos, educación y proyectos de electrónica accesible para principiantes, artistas, diseñadores y *makers*.

1.2. Familia y variantes

■ Entry-Level / Clásico:

- **Arduino UNO R3:** ATmega328P, 16 MHz, 32 KB Flash, 2 KB SRAM. La placa más popular de toda la familia.
- **Arduino UNO R4 Minima:** Renesas RA4M1 Cortex-M4, 48 MHz, 256 KB Flash, 32 KB SRAM, 8 KB EEPROM. Primera UNO de 32 bits.
- **Arduino UNO R4 WiFi:** Mismas prestaciones que la Minima pero con Wi-Fi y Bluetooth integrados mediante ESP32-S3.
- **Arduino Nano:** ATmega328P en formato compacto *breadboard-friendly*.
- **Arduino Nano Every:** ATmega4809 con más memoria que el Nano clásico.
- **Arduino Mega 2560:** ATmega2560, 256 KB Flash, 8 KB SRAM, 54 pines digitales.
- **Arduino Leonardo:** ATmega32U4 con USB nativo HID, 32 KB Flash, 2.5 KB SRAM.
- **Arduino Micro:** ATmega32U4, versión compacta del Leonardo.

■ IoT:

- **Arduino Nano ESP32:** SoC ESP32-S3 con Wi-Fi y Bluetooth integrados.
- **Arduino MKR WiFi 1010:** SAMD21 con módulo u-blox NINA-W10.
- **Arduino MKR WAN 1300/1310:** SAMD21 con LoRa.

■ AI / Industrial:

- **Arduino Portenta H7:** STM32H747 *dual-core* M7+M4, 2 MB Flash, 1 MB RAM, Wi-Fi, BT, cámara, USB-C.
- **Arduino GIGA R1 WiFi:** STM32H7 con Wi-Fi, BT y 76 pines de E/S.
- **Arduino Nicla Series:** Placas compactas para *machine learning* y fusión de sensores.

- **Diferencias principales:** El procesamiento va de 8 bits AVR a 32 bits ARM. La conectividad varía desde modelos sin inalámbrico hasta aquellos con Wi-Fi y BT. La memoria va de 2 KB SRAM (UNO R3) a 1 MB (Portenta H7). El precio oscila entre 4 USD (clones) y más de 100 USD (Portenta).

1.3. Microprocesador / Microcontrolador

Tomando como referencia el Arduino UNO R3, el microcontrolador es el ATmega328P, fabricado por Microchip Technology, con arquitectura AVR RISC de 8 bits, un solo núcleo y 16 MHz de frecuencia. En la gama alta, el Portenta H7 usa el STM32H747 de STMicroelectronics, con arquitectura ARM Cortex-M7 más Cortex-M4 en configuración *dual-core*: el M7 llega a 480 MHz y el M4 a 240 MHz.

1.4. Memoria

El UNO R3 cuenta con 32 KB de Flash, 2 KB de SRAM y 1 KB de EEPROM interna, sin soporte de memoria externa. El UNO R4 Minima ofrece 256 KB de Flash, 32 KB de SRAM y 8 KB de EEPROM. El Nano ESP32 dispone de 16 MB de Flash vía SPI y 512 KB de SRAM, con PSRAM opcional. El Portenta H7 integra 2 MB de Flash interna más 16 MB QSPI externa, junto con 1 MB de RAM interna y 8 MB de SDRAM externa.

1.5. Interfaces de comunicación

- **UART:** 1 puerto en UNO R3, hasta 3 en Mega y Portenta.
- **SPI:** 1 bus con pines dedicados MOSI, MISO, SCK y SS.
- **I²C:** 1 bus en pines A4 (SDA) y A5 (SCL).
- **USB:** USB-B en UNO R3, USB nativo HID en Leonardo, USB-C en Portenta H7.
- **CAN:** No disponible en modelos *entry-level*; presente en Portenta H7 mediante FD-CAN.
- **Ethernet:** No nativo; requiere *shield* W5100 o W5500.
- **Wi-Fi:** Ausente en UNO R3; integrado en UNO R4 WiFi, Nano ESP32, MKR WiFi y Portenta H7.
- **Bluetooth:** Ausente en UNO R3; integrado con BLE 5.1 en UNO R4 WiFi, Portenta H7 y Nano ESP32.
- **ADC:** 6 canales de 10 bits en UNO R3, 14 bits en UNO R4, 16 bits en Portenta.
- **DAC:** No en UNO R3; 1 canal de 12 bits en UNO R4 Minima; 2 DAC en Portenta.
- **PWM:** 6 pines en UNO R3, hasta 16 canales en Mega.
- **GPIO:** 14 digitales y 6 analógicos en UNO R3; más de 76 en GIGA R1.

1.6. Software de desarrollo (IDE)

- **IDE Oficial:** Arduino IDE 2.x (basado en Eclipse, Windows/macOS/Linux) y Arduino Web Editor en la nube.
- **IDEs Alternativos:** PlatformIO (VS Code / CLion), Keil MDK (Portenta H7), SEGGER Embedded Studio, CLion con plugin de Arduino.
- **Simuladores:** Tinkercad Circuits (Autodesk), Wokwi, SimulIDE, Proteus.

1.7. Lenguajes de programación soportados

El lenguaje principal es C y C++ en lo que se conoce como “Arduino Language”, una versión simplificada de C++. El chip admite además MicroPython, Lua mediante OpenMV IDE para la Nicla Vision, JavaScript y TypeScript con Espruino, Python en el Portenta, y Rust de forma experimental.

1.8. Sistema operativo

Las placas *entry-level* y *mid-range* no utilizan sistema operativo: funcionan en modo *bare-metal*. Entre los RTOS soportados están FreeRTOS, disponible para AVR y ARM; Mbed OS para Portenta H7 y MKR; Zephyr RTOS para Portenta H7; y Arduino_Threads, basado en Mbed. Ninguna de las placas clásicas de Arduino ejecuta Linux.

1.9. Aplicaciones más comunes

Arduino se usa en automatización del hogar, robótica educativa y amateur, y prototipado rápido de hardware. También aparece en arte interactivo, sensores IoT de bajo costo, y control de motores y servomotores. Las placas Nano y MKR se emplean en *wearables*, mientras que en educación STEM se usa ampliamente para estaciones meteorológicas y proyectos *maker* y DIY en general.

2. ESP32

2.1. Descripción general

2.2. Descripción general

El ESP32 es un SoC (*System-on-Chip*) fabricado por Espressif Systems de Shanghái, China, lanzado en 2016 como sucesor del ESP8266. Está orientado a aplicaciones IoT, dispositivos conectados en red, *wearables* y automatización industrial. Su bajo costo, bajo consumo y la integración de Wi-Fi y Bluetooth en un solo chip lo distinguen del resto de SoCs similares.



Figura 2: ESP32

2.3. Familia y variantes

- **Serie principal (Xtensa LX6/LX7):**
 - **ESP32 (clásico):** *Dual-core* Xtensa LX6 hasta 240 MHz, Bluetooth Classic + BLE. Es el más versátil y con el ecosistema de librerías más amplio.
 - **ESP32-S2:** *Single-core* LX7 a 240 MHz, USB OTG nativo, sin Bluetooth Classic.
 - **ESP32-S3:** *Dual-core* LX7 a 240 MHz, BLE 5.0, aceleración para *edge AI* y USB OTG.
 - **ESP32-C3:** *Single-core* RISC-V a 160 MHz, BLE 5.0 + Wi-Fi. Es el modelo de bajo costo de la familia.
 - **ESP32-C6:** *Single-core* RISC-V a 160 MHz, Wi-Fi 6 (802.11ax), BLE 5.3, Zigbee y Thread.
 - **ESP32-H2:** RISC-V, solo 802.15.4 (Zigbee/Thread) + BLE, sin Wi-Fi.
- **Módulos más comunes:**
 - **ESP32-WROOM-32:** Módulo estándar con antena integrada y 4 MB de Flash, el más utilizado.
 - **ESP32-WROVER:** Similar al WROOM-32 pero con 8 MB de PSRAM adicional.
 - **ESP32-C3-MINI:** Versión compacta del C3.
- **Diferencias principales:** El núcleo va desde LX6 (clásico) pasando por LX7 (S2/S3) hasta RISC-V (C3/C6/H2). Los protocolos inalámbricos varían: Wi-Fi + BT Classic + BLE en el clásico, solo BLE en S2/C3, y Wi-Fi 6 en C6. El USB nativo está presente en S2, S3 y C3, mientras que el clásico requiere un chip externo. En cuanto al precio, el C3 es el más económico y el S3 el más potente para aplicaciones de inteligencia artificial.

2.4. Microprocesador / Microcontrolador

El ESP32 clásico (ESP32-D0WD) utiliza el chip Tensilica Xtensa LX6 de Espressif Systems, una arquitectura Xtensa de 32 bits basada en RISC modificado. Cuenta con dos núcleos configurables como *dual-core* o *single-core*, frecuencia ajustable hasta 240 MHz (80, 160 o 240 MHz) y está fabricado en proceso TSMC de 40 nm de ultra bajo consumo.

2.5. Memoria

El ESP32 WROOM-32 dispone de 4 MB de Flash externa SPI y 520 KB de SRAM interna (448 KB para datos, 64 KB para instrucciones y 8 KB para RTC). El ESP32-WROVER añade 8 MB de PSRAM. El ESP32-S3 admite entre 4 y 16 MB de Flash externa, 512 KB de SRAM interna y hasta 8 MB de PSRAM mediante SPI octal. La EEPROM no existe físicamente; se emula en Flash usando la biblioteca NVS (*Non-Volatile Storage*).

2.6. Interfaces de comunicación

- **UART:** 3 interfaces (UART0, UART1, UART2) con velocidades de hasta 5 Mbps y soporte RS232/RS485.
- **SPI:** 3 buses (SPI, HSPI, VSPI) en modo *master* y *slave*, con modos 0 a 4.
- **I²C:** 2 buses con pines configurables mediante GPIO Matrix, soporte para hasta 112 dispositivos por bus.
- **USB:** No nativo en el clásico (requiere chip CP2102/CH340 externo); USB 2.0 OTG nativo en ESP32-S2, S3 y C3.
- **CAN:** TWAI (*Two-Wire Automotive Interface*) compatible con ISO 11898-1 (CAN 2.0).
- **Ethernet:** MAC Ethernet con soporte para PHY externo (LAN8720), DMA e IEEE 1588.
- **Wi-Fi:** IEEE 802.11 b/g/n a 2.4 GHz hasta 150 Mbps con soporte WPA3 en S3 y C6; Wi-Fi 6 disponible en ESP32-C6.
- **Bluetooth:** Clásico v4.2 + BLE 4.2 en el clásico; BLE 5.0 en C3, S2 y S3; BLE 5.3 en C6.
- **ADC:** 2 unidades SAR de 12 bits con 18 canales de entrada analógica. El ADC2 no está disponible cuando Wi-Fi está activo.
- **DAC:** 2 canales de 8 bits en los pines GPIO25 y GPIO26, solo en ESP32 clásico y S2.
- **PWM:** LED PWM Controller con 16 canales independientes y Motor Control PWM (MCPWM) con 2 unidades de 6 canales cada una.

- **GPIO:** 34 pines programables en el clásico, 43 en S3 y 30 en C3, con soporte de pullup/pulldown, interrupción por flanco/nivel, touch capacitivo en 10 pines y wakeup desde *deep sleep*.

2.7. Software de desarrollo (IDE)

- **IDE Oficial:** ESP-IDF (Espressif IoT Development Framework), framework nativo con acceso completo al hardware basado en C/C++. Disponible como plugin para VS Code y Eclipse.
- **IDEs Alternativos:** Arduino IDE mediante ESP32 Arduino Core (soporte oficial de Espressif), PlatformIO (VS Code, muy popular con soporte para ESP-IDF y Arduino), Thonny y uPyCraft para MicroPython, CLion con PlatformIO.
- **Simuladores:** Wokwi (simulador web muy completo con soporte para sensores, displays y Wi-Fi simulado), QEMU (experimental, vía ESP-IDF).

2.8. Lenguajes de programación soportados

C y C++ son los lenguajes principales mediante ESP-IDF y Arduino Core. También se soportan MicroPython (firmware oficial de Espressif), CircuitPython (parcial en algunas variantes), Lua con firmware NodeMCU, JavaScript con Espruino y Moddable SDK, C# con .NET nanoFramework (soporte oficial para ESP32-S3), Java con MicroEJ VEE, comandos AT para usar el chip como módulo de comunicación externo, y Rust con soporte creciente mediante esp-hal y esp-rs.

2.9. Sistema operativo

El uso de sistema operativo es opcional: puede funcionar en modo *bare-metal* o con RTOS. FreeRTOS viene integrado por defecto en ESP-IDF y se usa internamente para gestionar las pilas de Wi-Fi y Bluetooth. Hay soporte oficial de Zephyr RTOS para los modelos C3 y S3, NuttX de forma experimental, y ESP-ADF (basado en FreeRTOS) para aplicaciones de audio. El ESP32 no puede ejecutar Linux por carecer de MMU.

2.10. Aplicaciones más comunes

Entre los usos más frecuentes del ESP32 están los dispositivos IoT con conectividad Wi-Fi y Bluetooth, nodos de telemetría y monitoreo remoto, sistemas de control de acceso con BLE o Wi-Fi, cámaras IP de bajo costo (ESP32-CAM), *wearables* con BLE, automatización industrial mediante CAN y Ethernet, implementación de protocolos como MQTT, HTTP y WebSockets, *thin clients* y VNC remotos, sistemas de pago IoT, y control domótico con asistentes de voz locales.

3. STM32

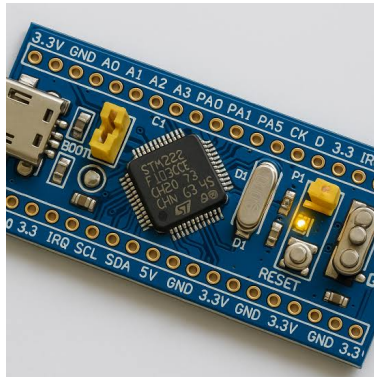


Figura 3: STM32

3.1. Descripción general

STM32 es una familia de microcontroladores de 32 bits de grado industrial fabricada por STMicroelectronics, con sede entre Francia e Italia. El primer chip, el STM32F103, se lanzó en 2007. Está orientada a control en tiempo real, sistemas de baja potencia y aplicaciones automotrices, industriales y médicas, y toda la familia se basa en núcleos ARM Cortex-M. Según la encuesta de Eclipse Foundation de 2024, el 34 % de los desarrolladores *embedded* profesionales usa STM32.

3.2. Familia y variantes

■ Gama de Entrada y Propósito General (*Entry-level* / *Mainstream*):

- **STM32C0:** Basado en Cortex-M0+ corriendo hasta 48 MHz; es la opción más económica de la familia.
- **STM32F0:** Equipado con Cortex-M0 hasta 48 MHz y un almacenamiento de 16 a 256 KB de Flash.
- **STM32F1:** Basado en Cortex-M3 hasta 72 MHz; el modelo F103C8T6, apodado “Blue Pill”, es el más conocido en el entorno *maker*.
- **STM32F3:** Incorpora núcleo Cortex-M4F con unidad de punto flotante (FPU) y capacidades DSP avanzadas.
- **STM32G0:** Sucesor moderno de la serie F0, implementa Cortex-M0+ con frecuencias de hasta 64 MHz.

■ Serie de Alto Rendimiento (*High-performance*):

- **STM32F4:** Basado en Cortex-M4F hasta 180 MHz con FPU; fue la primera serie en incorporar instrucciones DSP.
- **STM32F7:** Cuenta con núcleo Cortex-M7 hasta 216 MHz e interfaz nativa USB OTG HS.
- **STM32H7:** La serie más potente, con configuración de doble núcleo (*dual-core*) Cortex-M7 y Cortex-M4 que alcanza frecuencias de hasta 550 MHz.
- **Ultra Bajo Consumo (*Ultra-low-power*):**
 - **STM32L0:** Consumo de apenas 87 $\mu\text{A}/\text{MHz}$.
 - **STM32L4:** Modos de ahorro extremo: el modo *stop* consume solo 2 μA .
 - **STM32U5:** 19 $\mu\text{A}/\text{MHz}$ en modo activo, con periféricos LPBAM.
- **Conectividad Inalámbrica Integrada:**
 - **STM32WB:** Arquitectura de doble núcleo (Cortex-M4 y Cortex-M0+) dedicada a radios, con soporte BLE 5.4, Zigbee y Thread.
 - **STM32WL:** Variante especializada en redes de largo alcance, con modulaciones LoRa, FSK y BPSK.

3.3. Microprocesador / Microcontrolador

Tomando como referencia el STM32F4, el microcontrolador es fabricado por STMicroelectronics con arquitectura ARM Cortex-M4F de 32 bits, Thumb-2, DSP y FPU, un solo núcleo y frecuencia entre 84 y 180 MHz. El STM32H7 alcanza 550 MHz con configuración *dual-core* M7 y M4. La familia completa abarca desde Cortex-M0 hasta Cortex-M85.

3.4. Memoria

- **STM32F103C8T6 (Blue Pill):**
 - **Flash:** 64 KB para almacenamiento de programa.
 - **SRAM:** 20 KB de memoria de acceso aleatorio estática.
 - **EEPROM:** No la incluye de forma nativa, pero puede emularse por software usando la memoria Flash.
- **STM32F407VGT6 (Discovery Board):**
 - **Flash:** 1 MB de espacio de almacenamiento principal.
 - **SRAM:** 192 KB de propósito general, más 64 KB de memoria CCM (*Core Coupled Memory*) de alta velocidad.
- **STM32H743:**

- **Flash:** 2 MB en arquitectura de doble banco (*dual-bank*).
- **SRAM:** 1 MB interna de alto rendimiento.
- **STM32L4R9 (Ultra-low-power):**
 - **Flash:** 2 MB dedicados a código y datos.
 - **SRAM:** 640 KB optimizados para bajo consumo.
- **Memoria externa:** admite memorias externas mediante periféricos SDRAM, QSPI (*Quad-SPI*) y FMC (*Flexible Memory Controller*), y puede direccionar hasta varios gigabytes de almacenamiento según el bus y el encapsulado.

3.5. Interfaces de comunicación

- **Interfaces de Comunicación Serial:**
 - **USART / UART:** Hasta seis interfaces con soporte para RS232, RS485 e IrDA.
 - **SPI / I²S:** Tres buses de alta velocidad con soporte para audio digital I²S.
 - **I²C:** Tres buses con soporte *fast mode plus* a 1 MHz.
- **Conectividad de Red y Alta Velocidad:**
 - **USB 2.0 OTG:** Compatibilidad con Full Speed y High Speed (este último requiere un chip PHY externo).
 - **Controladores CAN:** Dos controladores CAN 2.0B (con soporte para CAN FD en las series más recientes).
 - **Ethernet MAC:** Conectividad 10/100 Mbps con DMA integrado y soporte para el estándar de sincronización IEEE 1588 (requiere PHY externo).
- **Conversión Analógica:**
 - **ADC:** Entre uno y tres convertidores con resolución de 12 a 16 bits y hasta 24 canales de entrada.
 - **DAC:** Dos canales de 12 bits, compatibles con transferencia por DMA.
- **Control y Temporización:**
 - **Modulación PWM:** Basada en los temporizadores avanzados TIM1 y TIM8 (4 canales cada uno más salidas complementarias), con más de 16 canales totales en el sistema.
- **Pines de Propósito General:**
 - **GPIO:** Entre 51 y 140 pines de entrada/salida según el encapsulado.

3.6. Software de desarrollo (IDE)

El IDE oficial es STM32CubeIDE, basado en Eclipse e integrado con STM32CubeMX para la generación gráfica de código de inicialización HAL; es gratuito y multiplataforma. Como alternativas se usan Keil MDK con μ Vision5, IAR Embedded Workbench en el ámbito industrial, PlatformIO para VS Code y CLion, Arduino IDE mediante el core STM32duino, y Mbed Studio. Los simuladores disponibles son Proteus Professional, QEMU con soporte parcial, y Renode.

3.7. Lenguajes de programación soportados

El lenguaje principal es C mediante los controladores HAL y LL de ST, junto con C++ y Assembly ARM para rutinas críticas. El chip admite además MicroPython para las series F4, F7 y H7, Rust con soporte activo mediante stm32-hal y knurling-rs, y Ada con SPARK para aplicaciones de seguridad crítica.

3.8. Sistema operativo

- **Bare-metal:** sin sistema operativo, habitual en los microcontroladores de gama baja.
- **RTOS:** recomendado para proyectos de complejidad media o alta. FreeRTOS es la opción más usada y viene integrada de forma nativa en STM32CubeMX. También hay soporte para Zephyr RTOS, ChibiOS/RT y Mbed OS (este último común en las placas Nucleo).
- **RTOS certificados:** SAFERTOS cuenta con certificaciones de seguridad crítica bajo IEC 61508 SIL3 e ISO 26262 ASILD. También hay compatibilidad con μ C/OS-III y Azure RTOS ThreadX.
- **Linux:** solo disponible en las variantes con microprocesador STM32MP1 y STM32MP2, que usan núcleos Cortex-A. Las series tradicionales de microcontroladores (F, G, H, L y U) no admiten Linux y quedan limitadas a RTOS o *bare-metal*.

3.9. Aplicaciones más comunes

STM32 se utiliza en control de motores y variadores de frecuencia, sistemas de medición industrial como PLC y SCADA, dispositivos médicos como monitores de signos vitales y bombas de infusión, sistemas automotrices como ECU, airbag y BMS de baterías, impresoras 3D y CNC, electrodomésticos inteligentes, sistemas de audio digital, robótica de precisión, *gateways* industriales con Ethernet y CAN, y aplicaciones de seguridad funcional certificadas bajo IEC 61508.



Figura 4: Raspberry Pi

4. Raspberry Pi

4.1. Descripción general

Raspberry Pi es una línea de computadoras de placa única fabricada por Raspberry Pi Foundation y Raspberry Pi Ltd. del Reino Unido, lanzada en febrero de 2012 con el modelo Raspberry Pi 1 Model B. Se diseñó originalmente para educación en informática, y con el tiempo se convirtió en una plataforma versátil para IoT, *media centers*, servidores caseros, robótica y sistemas embebidos que requieren Linux completo. La línea también incluye la serie Pico, microcontroladores sin sistema operativo.

4.2. Familia y variantes

- **Serie Principal (Soporte Completo para Linux):**
 - **Raspberry Pi 1 B+:** Equipado con un procesador ARM11 a 700 MHz y una memoria RAM de 512 MB.
 - **Raspberry Pi 2 B:** Incorpora un procesador Cortex-A7 *quad-core* a 900 MHz y 1 GB de memoria RAM.
 - **Raspberry Pi 3 B:** Primer modelo en integrar conectividad inalámbrica Wi-Fi y Bluetooth nativa, basado en un núcleo Cortex-A53 a 1.2 GHz.
 - **Raspberry Pi 4 B:** El modelo más popular de la serie; utiliza un procesador Cortex-A72 de hasta 1.8 GHz y configuraciones de memoria de hasta 8 GB de RAM LPDDR4.
 - **Raspberry Pi 5:** Basado en un procesador Cortex-A76 *quad-core* a 2.4 GHz, admite hasta 8 GB de RAM LPDDR4X e incluye un conector físico PCIe Gen 2.
- **Serie de Tamaño Ultra-reducido (Zero):**
 - **Raspberry Pi Zero 2 W:** Variante compacta equipada con un núcleo Cortex-A53 *quad-core* a 1 GHz y 512 MB de memoria RAM.
- **Serie para Integración Industrial (Compute Module):**

- **CM4 y CM5:** Módulos que replican las especificaciones técnicas del Pi 4 y Pi 5 respectivamente, en un factor de forma tipo SO-DIMM orientado a diseños de producción de hardware personalizados.

- **Serie de Microcontroladores (Pico):**

- **Raspberry Pi Pico / Pico W:** Basados en el chip propietario RP2040, arquitectura *dual-core* Cortex-M0+ a 133 MHz. La versión W añade conectividad inalámbrica.
- **Raspberry Pi Pico 2 / Pico 2 W:** Basados en el microcontrolador RP2350, a 150 MHz, que permite elegir entre núcleos Cortex-M33 o núcleos RISC-V Hazard3 en tiempo de ejecución.

4.3. Microprocesador / Microcontrolador

El Raspberry Pi 5 utiliza el chip BCM2712 de Broadcom con arquitectura ARM Cortex-A76 de 64 bits ARMv8.2-A, cuatro núcleos a 2.4 GHz y GPU VideoCore VII a 800 MHz. El Raspberry Pi Pico 2 emplea el RP2350 con arquitectura seleccionable entre ARM Cortex-M33 dual o RISC-V Hazard3 dual, dos núcleos a 150 MHz.

4.4. Memoria

El Raspberry Pi 5 dispone de 4 u 8 GB de RAM LPDDR4X, utiliza microSD o SSD NVMe mediante HAT+ como almacenamiento, y cuenta con EEPROM de arranque en SPI. El Pi 4 ofrece entre 1 y 8 GB de RAM LPDDR4 con almacenamiento en microSD, USB o SSD externo. El Pi Pico 2 cuenta con 520 KB de SRAM y 4 MB de Flash SPI, sin EEPROM.

4.5. Interfaces de comunicación

- **Comunicación y Conectividad (Pi 4 y Pi 5):**

- **Buses Seriales:** Dos interfaces UART, dos buses SPI y dos buses I²C nativos.
- **Puertos USB:** Dos puertos USB 3.0 de alta velocidad y dos puertos USB 2.0 estándar.
- **Red y Conectividad Inalámbrica:** Puerto Gigabit Ethernet, Wi-Fi 802.11ac de doble banda (estándar desde el modelo Pi 3 B+) y Bluetooth 5.0.

- **Pines de Expansión y Señales (GPIO):**

- **Cabezal Estándar:** Conector de 40 pines GPIO para periféricos digitales.
- **Modulación PWM:** Dos canales PWM basados en hardware.
- **Salida de Audio Analógica:** Se genera mediante la modulación PWM, direccionada al *jack* de 3.5 mm.

- **Periféricos Faltantes:**

- **CAN Bus:** No tiene soporte integrado; requiere controladores externos como el chip MCP2515.
- **ADC:** No incluye convertidor analógico-digital interno; las señales analógicas de entrada necesitan integrados externos como el MCP3008.

4.6. Software de desarrollo (IDE)

Para los modelos SBC con Linux, Raspberry Pi OS incluye Thonny para Python, Geany y acceso a cualquier IDE disponible en Linux como VS Code, PyCharm y Arduino IDE. Para la serie Pico, el IDE oficial es Thonny para MicroPython y VS Code con el Pico SDK para C y C++, con alternativas como Arduino IDE mediante el core de Earle Philhower y PlatformIO. El simulador Wokwi soporta el Pi Pico con MicroPython y C.

4.7. Lenguajes de programación soportados

En los modelos SBC con Linux completo se puede usar prácticamente cualquier lenguaje disponible en Debian ARM64, siendo Python el más popular para GPIO, junto con C, C++, Java, JavaScript con Node.js, Ruby, Perl, Go, Rust y Bash. Para la serie Pico en modo *bare-metal* o RTOS se soportan MicroPython, CircuitPython, C y C++ con el Pico SDK, y Assembly ARM.

4.8. Sistema operativo

- **Modelos SBC:** al ser computadoras completas, necesitan un sistema operativo para funcionar. La distribución oficial es Raspberry Pi OS, basada en Debian GNU/Linux y disponible en versiones de 32 y 64 bits. También soportan Ubuntu (Server y Desktop), Manjaro, Kali Linux, Fedora IoT, openSUSE, Arch Linux ARM y Alpine Linux, además de Windows 11 IoT Enterprise, Android AOSP y Chrome OS Flex.
- **Serie Pico:** no usa sistema operativo; funciona en modo *bare-metal*. Opcionalmente puede integrar RTOS de código abierto como FreeRTOS o Zephyr RTOS para gestionar tareas multitarea.

4.9. Aplicaciones más comunes

En los modelos SBC con Linux las aplicaciones incluyen servidores caseros como NAS, Pi-hole, VPN y Home Assistant, *media centers* con Kodi y Plex, educación en programación, robótica con visión computacional usando OpenCV, estaciones meteorológicas, emulación de consolas retro con RetroPie, clusters de computación con Kubernetes, cámaras de seguridad inteligentes con IA y domótica. En la serie Pico se utilizan para sensores IoT de bajo costo, control de motores, robótica simple, teclados y dispositivos HID personalizados, y medición de temperatura y humedad.

5. Nordic nRF52

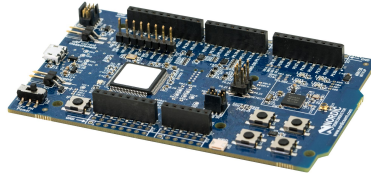


Figura 5: Nordic nRF52

5.1. Descripción general

El Nordic nRF52 es una familia de SoCs de ultra bajo consumo fabricada por Nordic Semiconductor de Trondheim, Noruega, lanzada en 2015 con el nRF52832. Está diseñada específicamente para comunicación inalámbrica de corto alcance, principalmente Bluetooth Low Energy, con soporte multiprotocolo para Zigbee, Thread, ANT, 802.15.4 y protocolos propietarios de 2.4 GHz. Es muy usado en *wearables*, dispositivos médicos portátiles y sensores IoT alimentados por batería.

5.2. Familia y variantes

- nRF52805: modelo básico con Cortex-M4 a 64 MHz, 192 KB de Flash y 24 KB de SRAM, orientado a aplicaciones simples y de bajo costo.
- nRF52810: versión de entrada con características similares al 52805, usado en soluciones BLE básicas.
- nRF52820: añade soporte para IEEE 802.15.4, permitiendo Zigbee y Thread en aplicaciones IoT.
- nRF52832: uno de los más populares, con Cortex-M4F, 512 KB de Flash y 64 KB de SRAM, soporte para BLE 5 y ANT.
- nRF52833: incorpora BLE 5.1 con *direction finding*, que mejora la localización mediante Bluetooth.
- nRF52840: el más avanzado de la familia, con 1 MB de Flash, 256 KB de SRAM, USB 2.0 nativo, NFC-A y soporte multiprotocolo (BLE, Thread, Zigbee y 802.15.4).
- nRF52-DK: kit de desarrollo basado en el nRF52832, usado ampliamente para aprendizaje y prototipado.
- nRF52840-DK: kit de desarrollo más completo de la familia, con todas las capacidades del nRF52840.

5.3. Microprocesador / Microcontrolador

Los chips nRF52 son diseñados por Nordic Semiconductor y fabricados en proceso TSMC de 55 nm. Utilizan arquitectura ARM Cortex-M4F de 32 bits con FPU en los modelos 52832, 52833 y 52840, y Cortex-M4 sin FPU en los modelos 52805, 52810 y 52820. Todos cuentan con un solo núcleo a 64 MHz.

5.4. Memoria

El nRF52832 dispone de 512 KB de Flash y 64 KB de SRAM, sin EEPROM aunque emulable mediante la biblioteca FDS de Nordic. El nRF52840 ofrece 1 MB de Flash y 256 KB de SRAM, con soporte QSPI para Flash externa de hasta 128 MB. El *SoftDevice*, que es el stack BLE de Nordic, ocupa aproximadamente entre 100 y 150 KB de Flash, dejando el resto para la aplicación del usuario.

5.5. Interfaces de comunicación

- UART: 2 interfaces de alta velocidad con EasyDMA (hasta 1 Mbps), usadas para comunicación serie eficiente.
- SPI: 3 interfaces maestro y 2 esclavo, utilizadas para comunicación rápida con sensores y memorias externas.
- I²C: 2 buses con EasyDMA para conectar múltiples periféricos con solo dos líneas.
- USB: USB 2.0 Full Speed nativo para conexión directa con computadoras y dispositivos host.
- Bluetooth LE: versiones 5.0, 5.1 y 5.2 con soporte para 2 Mbps, Long Range, Mesh y Direction Finding.
- IEEE 802.15.4: soporte para protocolos como Zigbee, Thread y Matter en redes IoT.
- ANT: protocolo inalámbrico de bajo consumo usado en sensores deportivos y wearables.
- NFC-A: comunicación de corto alcance para emparejamiento y lectura de etiquetas.

5.6. Software de desarrollo (IDE)

- nRF Connect SDK + VS Code: entorno oficial basado en Zephyr RTOS para desarrollo moderno en Nordic.
- SEGGER Embedded Studio: IDE gratuito optimizado para microcontroladores Nordic.
- Keil MDK μ Vision5: entorno profesional usado en sistemas embebidos ARM.

- IAR Embedded Workbench: IDE comercial con alta optimización de código para sistemas críticos.
- PlatformIO: plataforma multiplataforma para desarrollo embebido flexible y modular.
- Arduino IDE: soporte mediante BSP de Adafruit para prototipado rápido.
- Mbed Studio: entorno basado en ARM para desarrollo IoT simplificado.
- Herramientas Nordic: nRF Connect, nRF Toolbox y nRF Programmer para configuración, pruebas y flashing.
- Renode: herramienta de emulación parcial usada para pruebas sin hardware físico.

5.7. Lenguajes de programación soportados

- C: lenguaje principal utilizado en nRF5 SDK y nRF Connect SDK con Zephyr.
- C++: usado para estructuras más complejas y programación orientada a objetos.
- Assembly ARM: lenguaje de bajo nivel para optimización y control directo del hardware.
- MicroPython: implementación ligera para algunos dispositivos y prototipado rápido.
- Rust: lenguaje moderno con soporte mediante `nrf-hal` para sistemas seguros.
- JavaScript: usado de forma experimental con Espruino en placas compatibles.

5.8. Sistema operativo

- Bare-metal: ejecución directa del hardware sin sistema operativo para máximo rendimiento.
- SoftDevice: stack BLE propietario de Nordic que actúa como una capa tipo "soft RTOS".
- Zephyr RTOS: sistema operativo en tiempo real recomendado oficialmente e integrado en nRF Connect SDK.
- FreeRTOS: RTOS ligero usado en proyectos con el SDK legado.
- Mbed OS: sistema operativo orientado a IoT con soporte para dispositivos ARM.
- Linux: no soportado debido a la ausencia de unidad de gestión de memoria (MMU).

5.9. Aplicaciones más comunes

- Wearables: dispositivos como smartwatches, pulseras de actividad y auriculares BLE que requieren bajo consumo energético y conectividad constante.
- Dispositivos médicos portátiles: equipos como glucómetros, pulsioxímetros y ECG que transmiten datos biométricos en tiempo real de forma inalámbrica.
- Audífonos inteligentes: dispositivos de asistencia auditiva que utilizan Bluetooth LE para ofrecer audio de baja latencia y bajo consumo.
- Beacons Bluetooth: sistemas de localización en interiores que emiten señales periódicas para posicionamiento y seguimiento de dispositivos.
- Teclados, ratones y controladores inalámbricos: periféricos que requieren comunicación estable y eficiente con bajo consumo de batería.
- Sistemas de hogar inteligente: soluciones basadas en Thread, Matter y Zigbee para automatización y control de dispositivos del hogar.
- Rastreadores GPS y BLE de activos: dispositivos de seguimiento que combinan GPS y Bluetooth para localizar objetos o personas.
- Sensores industriales inalámbricos: nodos de monitoreo que envían datos de temperatura, vibración o estado en entornos industriales.
- Controladores para drones y robots: sistemas de control inalámbrico que requieren baja latencia para maniobras en tiempo real.
- Redes de sensores con BLE Mesh: redes distribuidas de sensores que se comunican entre sí para cubrir grandes áreas sin infraestructura cableada.

Fuentes consultadas

- Espressif Systems — *ESP32 Series Datasheet v5.2*. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Espressif Systems — *ESP-IDF Programming Guide*. <https://docs.espressif.com/projects/esp-idf/en/latest/>
- Nordic Semiconductor — *nRF52840 Product Specification v1.1*. https://infocenter.nordicsemi.com/pdf/nRF52840_PS_v1.1.pdf
- Nordic Semiconductor — *nRF52 Series Product Page*. <https://www.nordicsemi.com/Products/nRF52-Series-SoC>

- STMicroelectronics — *STM32 Family Overview*. <https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html>
- STMicroelectronics — *STM32CubeMX*. <https://www.st.com/en/development-tools/stm32cubemx.html>
- Raspberry Pi Foundation — *Raspberry Pi Product Pages*. <https://www.raspberrypi.com/products/>
- Raspberry Pi Foundation — *RP2350 Datasheet*. <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- Arduino — *Arduino Official Documentation*. <https://docs.arduino.cc/>
- Arduino — *Arduino Board Comparison*. <https://store.arduino.cc/pages/compare>
- Eclipse Foundation — *Embedded Developer Survey 2024*. <https://eclipse.dev/community/resources/eclipse-community-survey-2024/>
- PlatformIO — *PlatformIO Documentation*. <https://docs.platformio.org/>
- FreeRTOS — *FreeRTOS Official Site*. <https://www.freertos.org/>
- Zephyr Project — *Zephyr RTOS Documentation*. <https://docs.zephyrproject.org/>